

Deep Learning Optimized on Jean Zay

Profiler

IDRIS

BY NC SA

Benchmarking vs Profiling

Benchmarking vs Profiling

BENCHMARKING = How fast ?

- User metrics on the whole run: time / step, samples / s, tokens / s etc.
- No instrumentation, low impact
- Compare configurations

PROFILING = Why, and where ?

- Time and resources broken down per phase, op, kernel
- Instrumented, degraded performance
- Locate the bottleneck



Before talking about tools, we separate two approaches that are often mixed up: measuring performance, and understanding where it comes from. The rest of the course relies on this distinction.

Benchmarking answers the question "how much?". It produces one global number: time per iteration, images per second. It is a black box with almost no overhead, and it is the ideal tool to compare two configurations: batch size, AMP, channels_last… This is exactly what the "check-up" of the previous labs did. But it does not tell why it is slow.

Profiling answers "where and why?". It breaks time and resources down by phase, by operation, by kernel. It is instrumented, so it slows the run down and slightly disturbs what it measures: we profile a few iterations after a warmup, never a full training run (it is useless, especially in deep learning).

The two form a loop: measure, profile to find the bottleneck, fix it, and measure again to check the gain, if any.

Avant de parler outils, on distingue deux démarches qu'on mélange souvent : mesurer la performance, et comprendre d'où elle vient. Toute la suite du cours repose sur cette distinction.

Le benchmarking répond à la question « combien ? ». On produit un chiffre global : temps par itération, images par seconde. C'est une boîte noire, quasiment sans surcoût, et c'est l'outil idéal pour comparer deux configurations : taille de batch, AMP, channels_last… C'est exactement ce que faisait le « contrôle technique » des TP précédents. Mais il ne dit pas pourquoi c'est lent.

Le profiling répond à « où et pourquoi ? ». Il découpe le temps et les ressources par phase, par opération, par kernel. Il est instrumenté, donc il ralentit l'exécution et perturbe un peu ce qu'il mesure : on profile quelques itérations après un warmup, jamais un entraînement complet (c'est inutile surtout en deep learning).

Les deux s'enchaînent en boucle : on mesure, on profile pour trouver le goulot, on corrige, et on remesure pour vérifier le gain eventuelle.

Key metrics

BENCHMARKING = How fast ?

Throughput	MFU	Time per step	Memory
samples / s: the number that matters	achieved FLOP/s ÷ GPU peak FLOP/s	Seconds per training iteration	Max GPU memory used

PROFILING = Why, and where ?

SM occupancy Tensor Cores	GPU utilization	Time breakdown	Bandwidth
Are the compute units well used ?	% of time kernel runs	Input wait, compute, communication, per op and kernel etc...	HBM, PCIe , interconnect

Before opening a profiler, you need to know which numbers to look at. Some tell whether it is fast, others explain why.

First row, the benchmarking metrics: they tell whether it is fast, without any instrumentation.

– Throughput, in images per second, is the number that really matters; time per iteration is its inverse.

– MFU (Model FLOPs Utilization) compares the FLOP/s actually reached with the theoretical peak of the GPU. Reference point: 40 to 50% is already a good result; Llama 3.1 reports 38 to 43%.

– Peak memory tells whether the model fits and how much room is left to increase the batch. This is what the lab check-up reports.

Second row, the profiling metrics: they explain the result.

– Time breakdown: waiting for data, compute, communication, then per operation and per kernel.

– GPU utilization is a trap: it only measures the fraction of time a kernel is running, not whether it computes efficiently. It can show 100% with 20% MFU. Cross-check it with power draw: a GPU that really computes gets close to 400 W on an A100.

– Occupancy and Tensor Core usage tell whether the compute units are well used.

– Three bandwidths not to confuse: HBM (≈ 2 TB/s on A100), NVLink (600 GB/s on A100), PCIe Gen4 (≈ 32 GB/s).

Avant d'ouvrir un profiler, il faut savoir quels chiffres regarder. Certains disent si c'est rapide, d'autres expliquent pourquoi.

Première ligne, les métriques de benchmarking : elles disent si c'est rapide, sans rien instrumenter.
– Le débit, en images par seconde, est le chiffre qui compte vraiment ; le temps par itération en est l'inverse.
– La MFU (Model FLOPs Utilization) rapporte les FLOP/s réellement atteints au pic théorique du GPU. Repère : 40 à 50 % est déjà un bon résultat ,Llama 3.1 annonce 38 à 43 %.
– Le pic mémoire dit si le modèle tient et quelle marge il reste pour augmenter le batch. C'est ce que reporte le contrôle technique des TP.

Deuxième ligne, les métriques de profiling : elles expliquent le résultat.
– La décomposition du temps : attente des données, calcul, communication, puis par opération et par kernel.
– L'utilisation GPU est un piège : elle mesure seulement la fraction du temps où un kernel tourne, pas s'il calcule efficacement. On peut afficher 100 % pour 20 % de MFU. La croiser avec la puissance consommée : un GPU qui calcule vraiment approche les 400 W sur un A100.
– L'occupancy et l'usage des Tensor Cores disent si les unités de calcul sont bien employées

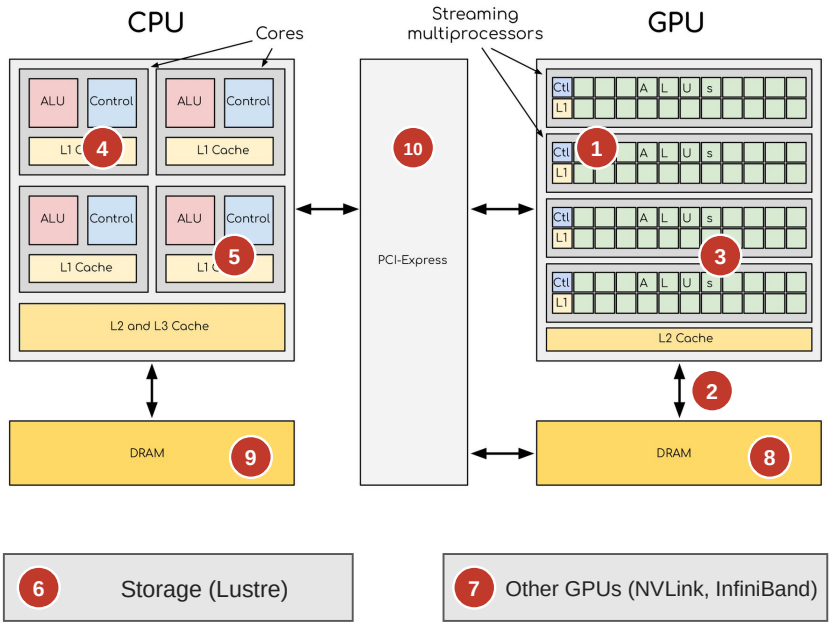
Bottlenecks

Now the bottlenecks, the "bounds". The idea: be able to name what limits speed, in order to choose the right tool and the right fix.

On passe aux goulots d'étranglement, les « bounds ». L'idée : savoir nommer ce qui limite la vitesse, pour choisir le bon outil et la bonne correction.

Where bottlenecks live

https://enccs.github.io/gpu-programming/2-gpu-ecosystem/#id1



Does the GPU compute well ?

1. Compute
2. Memory bandwidth
3. Latency / occupancy

Is the GPU fed ?

4. Kernel launch
5. Data loading
6. Storage I/O
7. Communication

Does the hardware cap it ?

8. GPU memory (VRAM)
9. Host memory (RAM)
10. PCIe transfers

This diagram places each bottleneck in the hardware. On the left the CPU and its memory, in the middle the PCIe link, on the right the GPU, its multiprocessors (SMs) and its HBM memory.

We sort them with three questions:

1. Does the GPU compute efficiently? Compute, memory bandwidth, latency: everything happens inside the GPU.
2. Is the GPU fed? Kernel launch, data loading, disk reads, communication: the GPU waits for something coming from elsewhere.
3. Does the hardware hit a ceiling? Capacity of the VRAM, of the RAM, throughput of the PCIe link.

Two ideas to remember. The same training run changes bottleneck over time: forward limited by compute, all-reduce limited by communication, start of step limited by data. And fixing one bottleneck often reveals another: it is a sign that we are getting closer to the hardware limit, which in itself is not always a bad sign.

Ce schéma situe chaque goulot dans le matériel. À gauche le CPU et sa mémoire, au milieu le lien PCIe, à droite le GPU, ses multiprocesseurs (SM) et sa mémoire HBM.

On les range selon trois questions :

1. Le GPU calcule-t-il efficacement ? Compute, bande passante mémoire, latence : tout se passe dans le GPU.
2. Le GPU est-il alimenté ? Lancement des kernels, chargement des données, lecture disque, communication : le GPU attend quelque chose venant d'ailleurs.
3. Le matériel plafonne-t-il ? Capacité de la VRAM, de la RAM, débit du lien PCIe.

Deux idées à retenir. Un même entraînement change de goulot au fil du temps : forward limité par le calcul, all-reduce limité par la communication, début de pas limité par les données. Et corriger un goulot en révèle souvent un autre : c'est le signe qu'on progresse vers la limite du matériel, en soit ce n’est pas toujours un mauvais signe.

1 · Compute-bound

DEFINITION

More math than the GPU can do.

CAUSE

Big matrix multiplications and convolutions, or FP32 math that skips the Tensor Cores

CONSEQUENCE

Good near peak. On the FP32 path, most of the hardware is wasted

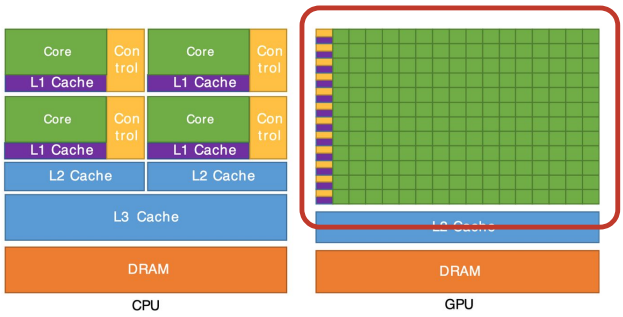
SYMPTOMS

High compute throughput, lower memory throughput.

FIX

Mixed precision (bf16 / fp16), dimensions multiple of 8, fused kernels

https://docs.nvidia.com/cuda/cuda-c-programming-guide/



Thousands of ALUs and Tensor Cores: all busy

Definition: the GPU has more computation to do than its arithmetic units can absorb.

On the diagram, the GPU is mostly this green grid: thousands of simple compute units, plus the Tensor Cores specialized in matrix products. When they are all busy, we are compute-bound.

Careful, this is not necessarily a problem: for a large matrix multiplication, being compute-bound near the peak is the goal. The bad case is being compute-bound on the wrong path: in pure FP32, the Tensor Cores are useless and we cap at about 19.5 TFLOP/s on A100, versus 312 TFLOP/s in FP16 on the Tensor Cores.

Symptoms: high compute throughput and lower memory throughput.

Fixes: mixed precision (that was TP1_2), dimensions in multiples of 8 so the Tensor Cores kick in, kernel fusion. If you are already at the peak, the only option left is to reduce the number of operations.

Définition : le GPU fait plus de calcul que ses unités arithmétiques ne peuvent en absorber.

Sur le schéma, le GPU, c'est surtout cette grille verte : des milliers d'unités de calcul simples, plus les Tensor Cores spécialisés dans les produits de matrices. Quand elles sont toutes occupées, on est compute-bound.

Attention, ce n'est pas forcément un problème : pour une grosse multiplication de matrices, être compute-bound près du pic, c'est l'objectif. Le mauvais cas, c'est d'être compute-bound sur le mauvais chemin : en FP32 pur, les Tensor Cores ne servent à rien et on plafonne à environ 19,5 TFLOP/s sur A100, contre 312 TFLOP/s en fp16 sur les Tensor Cores.

Symptômes : débit de calcul élevé et débit mémoire plus bas.

Corrections : précision mixte (c'était le TP1_2), dimensions multiples de 8 pour que les Tensor Cores s'activent, fusion de kernels. Si on est déjà au pic, il ne reste qu'à réduire le nombre d'opérations.

2 · Memory-bandwidth-bound

DEFINITION

The GPU waits for data from its own memory (HBM).

CAUSE

Few operations per byte: element-wise ops, normalizations, reductions, casts

CONSEQUENCE

The GPU looks busy, but MFU stays low

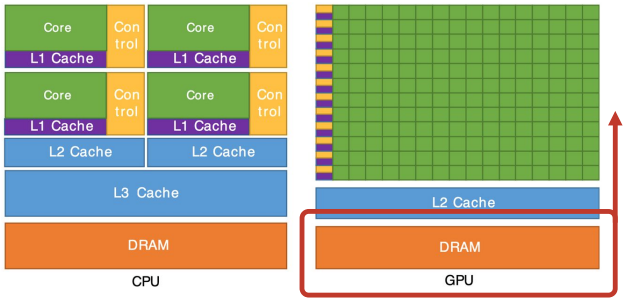
SYMPTOMS

High memory throughput, low compute throughput

FIX

Kernel fusion (torch.compile), smaller data types

https://docs.nvidia.com/cuda/cuda-c-programming-guide/



HBM ≈ 2 TB/s… still too slow to feed every ALU

Definition: the GPU spends its time waiting for data from its own memory, the HBM.

On the diagram, data must travel from the GPU's VRAM to the compute units. Even at about 2 TB/s on A100, this is too slow when each byte read is used for only one or two operations.

Typical causes: all operations with low arithmetic intensity. Activation functions, normalizations, additions, reductions, type conversions: we read a tensor, do almost nothing, and write it back.

Tricky consequence: the GPU looks busy, the displayed utilization is high, but MFU stays low, because the compute units are waiting.

Symptoms: high memory throughput and low compute throughput.

Fixes: fuse kernels so memory is read and written only once (torch.compile does it automatically for element-wise operations), FlashAttention (LLMs, not covered here), which avoids materializing the large attention matrix, and use smaller data types to move fewer bytes.

Définition : le GPU passe son temps à attendre les données venant de sa propre mémoire, la HBM.

Sur le schéma, les données doivent remonter de la VRAM du GPU vers les unités de calcul. Même à environ 2 To/s sur A100, c'est trop lent quand chaque octet lu ne sert qu'à une ou deux opérations.

Causes typiques : toutes les opérations à faible intensité arithmétique. Fonctions d'activation, normalisations, additions, réductions, conversions de type : on lit un tenseur, on fait presque rien, on l'écrit.

Conséquence piège : le GPU paraît occupé, l'utilisation affichée est haute, mais la MFU reste basse, parce que les unités de calcul attendent.

Symptômes : débit mémoire élevé et débit de calcul faible ; su

Corrections : fusionner les kernels pour ne lire et écrire la mémoire qu'une fois (torch.compile le fait automatiquement pour les opérations élément par élément), FlashAttention (LLM pas traité) qui évite de matérialiser la grande matrice d'attention, et réduire la taille des types pour déplacer moins d'octets.

3 · Latency / occupancy-bound

DEFINITION

Not enough parallel work to keep the GPU busy.

CAUSE

Tiny kernels, small batches, chains of dependent operations

CONSEQUENCE

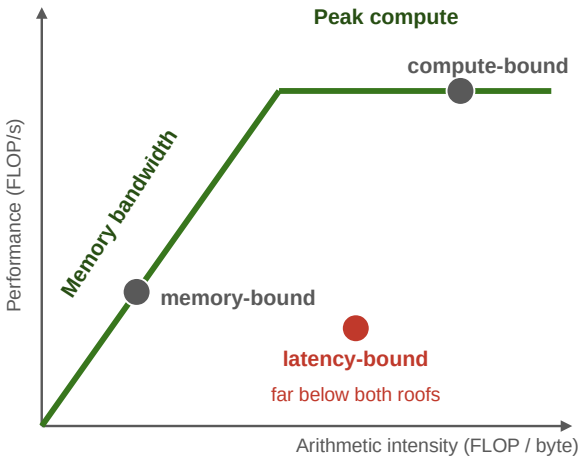
Neither compute nor memory is saturated

SYMPTOMS

Low occupancy, short kernels, both throughputs low

FIX

Larger batch, fuse small kernels



Definition: there is not enough parallel work to fill the GPU.

The diagram is a roofline, the tool that sums up the three GPU bottlenecks. On the x-axis, arithmetic intensity: how many operations per byte read. On the y-axis, the performance reached. The roof has two parts: the slope on the left, limited by memory bandwidth, and the plateau, limited by compute power. A memory-bound kernel sits on the slope, a compute-bound kernel on the plateau. A latency-bound kernel is far below both: it saturates neither.

Causes: tiny kernels, a batch that is too small, operations that depend on each other and cannot run in parallel.

This is what we saw with batch size 64 in the lab: the GPU is barely loaded, each kernel is too short to keep it busy.

Symptoms: low occupancy, very short kernels, both compute and memory throughput low.

Fixes: increase the batch, fuse small kernels.

Définition : il n'y a pas assez de travail en parallèle pour remplir le GPU.

Le schéma est un roofline, l'outil qui résume les trois goulots du GPU. En abscisse, l'intensité arithmétique : combien d'opérations par octet lu. En ordonnée, la performance atteinte. Le toit a deux pans : la pente de gauche, limitée par la bande passante mémoire, et le plateau, limité par la puissance de calcul. Un kernel memory-bound se trouve sur la pente, un kernel compute-bound sur le plateau. Un kernel latency-bound est loin en dessous des deux : il ne sature ni l'un ni l'autre.

Causes : des kernels minuscules, un batch trop petit, des opérations qui dépendent les unes des autres et ne peuvent pas tourner en parallèle.

C'est ce qu'on observait avec le batch de 64 dans le TP : le GPU est peu dense, chaque kernel est trop court pour l'occuper.

Symptômes : occupancy faible, kernels très courts, débits de calcul et de mémoire tous deux bas.

Corrections : augmenter le batch, fusionner les petits kernels

4 · Kernel-launch-bound

DEFINITION

The CPU cannot launch GPU kernels fast enough.

CAUSE

Python overhead, thousands of tiny operations

CONSEQUENCE

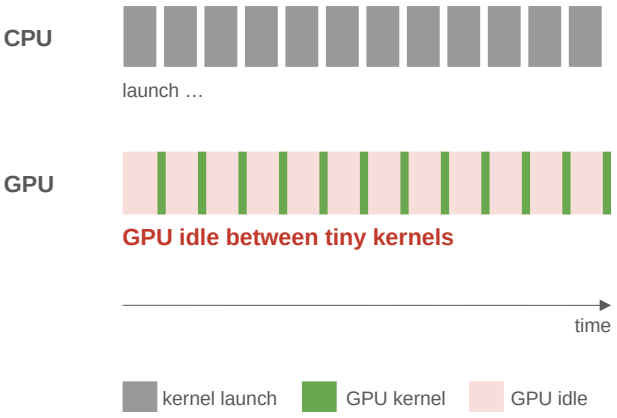
The GPU sits idle between kernels

SYMPTOMS

Gaps between kernels on the GPU timeline, CPU busy launching

FIX

torch.compile, bigger batches



Definition: the CPU cannot launch kernels fast enough to keep the GPU busy.

In eager mode, PyTorch launches each operation one by one, from Python. Each launch costs a few microseconds on the CPU side. If the launched kernel is even shorter than that, the GPU finishes it before the CPU has prepared the next one: it waits. On the timeline, the CPU row is full of launches, the GPU row is almost empty.

This happens with many small operations: models with many thin layers, small batches, Python code inside the loop.

Symptoms: in the PyTorch Profiler timeline opened in Perfetto, or in Nsight Systems, gaps between kernels on the GPU row, lined up with continuous launch activity on the CPU row.

Fixes: torch.compile also reduces the number of launches by fusing operations. And a bigger batch makes each kernel longer, so the launch is relatively cheaper.

Définition : le CPU n'arrive pas à lancer les kernels assez vite pour occuper le GPU.

En mode eager, PyTorch lance chaque opération une par une, depuis Python. Chaque lancement coûte quelques microsecondes côté CPU. Si le kernel lancé est encore plus court que ça, le GPU le termine avant que le CPU ait préparé le suivant : il attend. Sur la frise, la ligne CPU est pleine de lancements, la ligne GPU est presque vide.

Ce cas apparaît avec beaucoup de petites opérations : modèles avec beaucoup de couches fines, petits batchs, code Python dans la boucle.

Symptômes : dans la timeline du PyTorch Profiler ouverte dans Perfetto, ou dans Nsight Systems, des trous entre les kernels sur la ligne GPU, alignés avec une activité de lancement continue sur la ligne CPU.

Corrections : torch.compile réduit aussi le nombre de lancements en fusionnant les opérations. Et un batch plus gros rend chaque kernel plus long, donc le lancement relativement moins coûteux.

5 · Data-loading-bound

DEFINITION

The input pipeline cannot deliver batches fast enough.

CAUSE

Too few DataLoader workers, heavy CPU decoding and augmentation

CONSEQUENCE

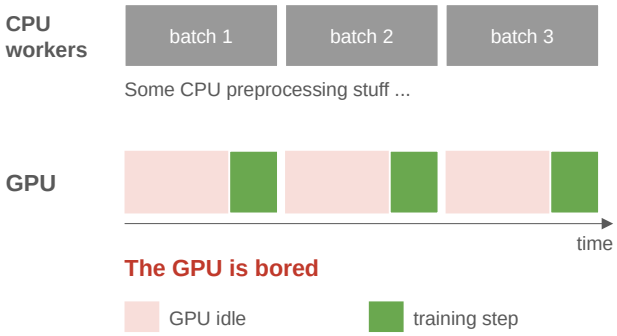
The GPU waits at the start of every step

SYMPTOMS

Long input wait, saw-tooth GPU utilization

FIX

More workers, pin_memory, prefetching, GPU augmentation (DALI)



Definition: the data pipeline does not deliver batches fast enough.

For ImageNet, for example, each image must be read, decoded (JPEG), cropped, flipped and normalized, on the CPU. With too few workers, preparing a batch takes longer than the training step on the GPU. On the timeline, the GPU computes quickly, then waits for the next batch.

It is the most common bottleneck in vision, and the first one to check. It is also what the lab shows with TraceML: with batch 64 and 4 workers, a large part of the step is spent in "Input Wait".

Symptoms: INPUT-BOUND verdict from TraceML, saw-tooth GPU utilization in nvidia-smi or CEEMS, gaps at the start of each step in the PyTorch Profiler timeline.

Fixes, from the simplest to the heaviest (see the DataLoader part on day 2): increase num_workers, enable pin_memory and prefetching, keep workers persistent; then move augmentation to the GPU (NVIDIA DALI); finally precompute or cache the preprocessed data.

Définition : le pipeline de données ne livre pas les batchs assez vite.

Pour ImageNet par exemple, chaque image doit être lue, décodée (JPEG), recadrée, retournée, normalisée, sur le CPU. Avec trop peu de workers, préparer un batch prend plus de temps que le pas d'entraînement sur le GPU. Sur la frise, le GPU calcule vite, puis attend le batch suivant.

C'est le goulot le plus fréquent en vision, et le premier à vérifier. C'est aussi ce que le TP montre avec TraceML : en batch 64 avec 4 workers, une grosse part du pas est passée en « Input Wait ».

Symptômes : verdict INPUT-BOUND de TraceML, utilisation GPU en dents de scie dans nvidia-smi ou CEEMS, trous au début de chaque pas dans la timeline du PyTorch Profiler.

Corrections, de la plus simple à la plus lourde (voir Dataloader jour 2 surtout) : augmenter num_workers, activer pin_memory et le prefetch, garder les workers persistants ; puis déplacer l'augmentation sur le GPU (NVIDIA DALI,) ; enfin précalculer ou mettre en cache les données prétraitées.

6 · Storage I/O-bound

DEFINITION

Reading the dataset from disk is the limit.

CAUSE

Millions of small files on a parallel file system (Lustre)

CONSEQUENCE

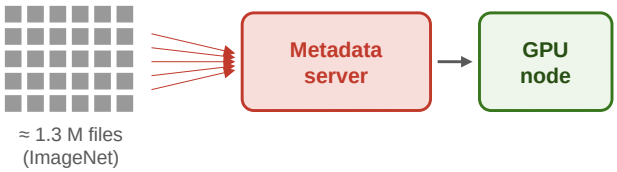
Metadata server saturated: slow first epoch, input stalls

SYMPTOMS

Long open / stat times, disk bandwidth not saturated

FIX

Sharded formats (WebDataset, HDF5): fewer, bigger files



Definition: reading data from disk is the limit, even before decoding.

On a parallel file system like Lustre, each opened file costs requests to the metadata server: open, read its attributes, close. ImageNet is about 1.3 million JPEG files of a few tens of kilobytes. The metadata server saturates long before the disks reach their bandwidth.

Consequences: a very slow first epoch, dataset indexing that takes minutes, jerky feeding of the GPU. And on a supercomputer this server is shared: a job that floods it also slows down other users.

Symptoms: a lot of time spent in open and stat, while disk bandwidth is not saturated. On the training side, it looks like a data-loading problem: TraceML shows a high Input Wait, for example.

Fix: group the small files into a few large files, "shards" (WebDataset tar archives, HDF5), read as a continuous stream. See day 3 of DLO-JZ.

Définition : c'est la lecture des données sur le disque qui limite, avant même le décodage.

Sur un système de fichiers parallèle comme Lustre, chaque fichier ouvert coûte des requêtes au serveur de métadonnées : ouvrir, lire ses attributs, fermer. ImageNet, c'est environ 1,3 million de fichiers JPEG de quelques dizaines de kilo-octets. Le serveur de métadonnées sature bien avant que les disques n'atteignent leur débit.

Conséquences : une première epoch très lente, l'indexation du dataset qui prend des minutes, des à-coups dans l'alimentation du GPU. Et sur un supercalculateur, ce serveur est partagé : un job qui l'inonde ralentit aussi les autres utilisateurs.

Symptômes : beaucoup de temps passé en open et stat, alors que la bande passante disque n'est pas saturée. Côté entraînement, ça ressemble à un problème de dataloading : TraceML montre un Input Wait élevé par exemple.

Correction : regrouper les petits fichiers dans quelques gros fichiers, des « shards » (WebDataset en archives tar, HDF5), qu'on lit en flux continu. Voir le jour 3 de DLO

7 · Communication-bound

DEFINITION

GPUs spend their time exchanging data instead of computing.

CAUSE

Gradient all-reduce (DDP, FSDP), slow links between nodes, no overlap

CONSEQUENCE

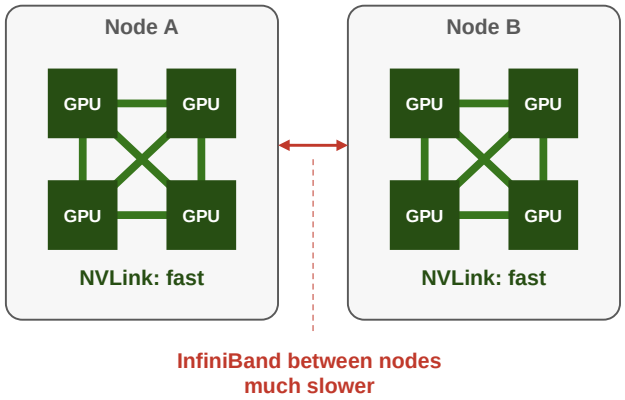
Adding GPUs brings less and less speed-up

SYMPTOMS

Long NCCL kernels, poor compute / comm overlap, sub-linear scaling

FIX

Overlap comm and compute, gradient accumulation, keep traffic on NVLink



Definition: the GPUs spend their time exchanging data instead of computing.

In data-parallel distributed training (DDP), each GPU computes its gradients, then all GPUs average their gradients at each step: this is the all-reduce, done by the NCCL library. With FSDP, the parameters themselves are exchanged as well.

Inside a node, GPUs are connected by NVLink, which is very fast (600 GB/s on A100). Between nodes, traffic goes through the InfiniBand network, which is much slower. An all-reduce is never faster than its slowest link.

Consequence: scaling efficiency drops. Doubling the number of GPUs no longer halves the time.

Symptoms: in Nsight Systems, NCCL kernels take a large part of the timeline; the communication share is high and the overlap with compute is low.

Fixes: overlap communication with compute (DDP already sends gradients in buckets during the backward pass), accumulate gradients to synchronize less often, and place the parallelism so that heavy traffic stays inside nodes. We will go deeper in the distributed part.

Définition : les GPU passent leur temps à échanger des données au lieu de calculer.

En entraînement distribué data-parallel (DDP), chaque GPU calcule ses gradients, puis tous les GPU font la moyenne de leurs gradients à chaque pas : c'est l'all-reduce, réalisé par la bibliothèque NCCL. En FSDP, on échange en plus les paramètres eux-mêmes.

À l'intérieur d'un nœud, les GPU sont reliés par NVLink, très rapide (600 Go/s sur A100). Entre nœuds, on passe par le réseau InfiniBand, nettement plus lent. Un all-reduce ne va jamais plus vite que son lien le plus lent.

Conséquence : l'efficacité du passage à l'échelle chute. Doubler le nombre de GPU ne divise plus le temps par deux.

Symptômes : dans Nsight Systems, les kernels NCCL prennent une grosse part de la timeli la part de communication est élevée et le recouvrement avec le calcul est faible.

Corrections : recouvrir la communication avec le calcul (DDP envoie déjà les gradients par paquets pendant le backward), accumuler les gradients pour synchroniser moins souvent, et placer le parallélisme pour garder le trafic lourd à l'intérieur des nœuds. On approfondira dans la partie distribuée.

8 · GPU memory capacity (VRAM)

DEFINITION

The model and its training state do not fit in GPU memory.

CAUSE

Parameters + gradients + optimizer states + activations > VRAM

CONSEQUENCE

CUDA out-of-memory, or workarounds that slow training down

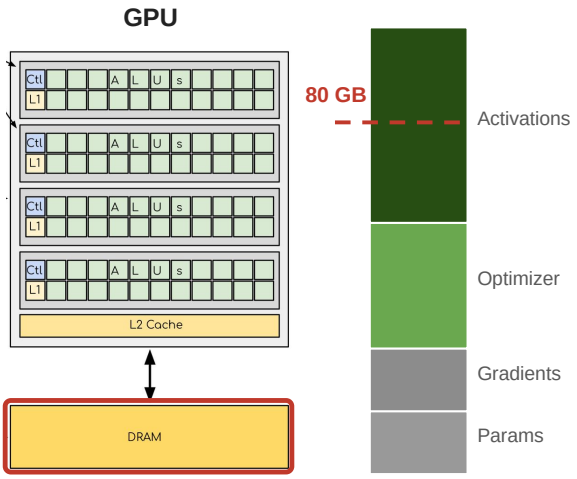
SYMPTOMS

OOM errors, peak memory close to capacity

FIX

AMP, activation checkpointing, FSDP / ZeRO sharding, gradient accumulation

https://enccs.github.io/gpu-programming/2-gpu-ecosystem/#id1



Definition: the model and all the state needed for training do not fit in GPU memory.

During training, VRAM does not hold only the model. There are the parameters, the gradients (same size), the optimizer states (twice the parameter size for Adam), and above all the activations kept for the backward pass, which grow with the batch size. On an A100, the ceiling is 80 GB.

Consequences: the CUDA out of memory error. Or, more insidious, workarounds that hurt performance: a tiny batch that makes the GPU latency-bound, or CPU offload that becomes PCIe-bound. It is a "cascading" bottleneck that triggers others.

Symptoms: the OOM error, a memory peak close to capacity. You can see it in the PyTorch Profiler memory timeline and in the Step Memory section of TraceML.

Fixes: mixed precision halves the size of the activations; activation checkpointing recomputes some activations instead of storing them; FSDP or ZeRO split parameters, gradients and optimizer states across GPUs; gradient accumulation simulates a large batch with several small ones.

Définition : le modèle et tout l'état nécessaire à l'entraînement ne tiennent pas dans la mémoire du GPU.

À l'entraînement, la VRAM ne contient pas que le modèle. Il y a les paramètres, les gradients (même taille), les états de l'optimiseur (deux fois la taille des paramètres pour Adam), et surtout les activations conservées pour le backward, qui grossissent avec la taille du batch. Sur un A100, le plafond est à 80 Go.

Conséquences : l'erreur CUDA out of memory. Ou, plus sournois, des contournements qui dégradent les performances : un batch minuscule qui rend le GPU latency-bound, ou de l'offload vers le CPU qui devient limité par le PCIe. C'est un goulot « en cascade » qui en déclenche d'autres.

Symptômes : l'erreur OOM, un pic mémoire proche de la capacité. On le voit dans la timeline mémoire du PyTorch Profiler, dans la section Step Memory de TraceML

Corrections : la précision mixte divise la taille des activations par deux ; l'activation checkpointing recalcule certaines activations au lieu de les stocker ; FSDP ou ZeRO répartissent paramètres, gradients et états de l'optimiseur entre GPU ; l'accumulation de gradients simule un gros batch avec plusieurs petits.

9 · Host memory capacity (RAM)

DEFINITION

The node's CPU memory runs out.

CAUSE

Whole dataset cached in RAM, memory leaks, workers duplicating Python data

CONSEQUENCE

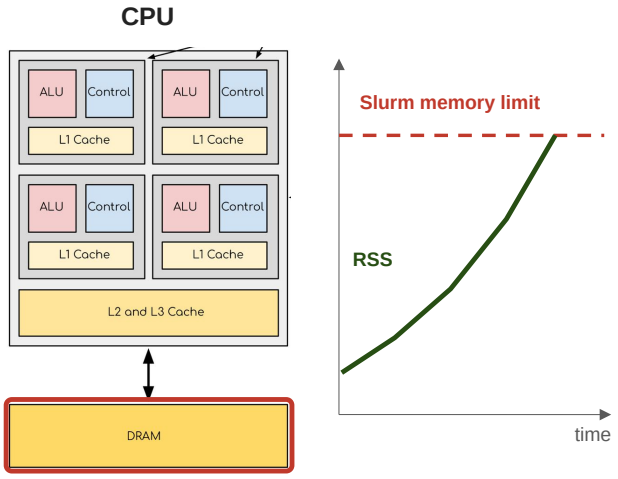
Job killed by Slurm (out of memory), often with no Python error

SYMPTOMS

Process memory keeps growing up to the job limit

FIX

Stream the data, fix leaks (.item(), .detach()), NumPy arrays over Python lists



Definition: the CPU memory, the node's RAM, is saturated.

On the diagram, we are now on the CPU side, in its DRAM. Three classic causes:

– loading or caching the whole dataset in RAM;

– a memory leak in the loop, for example accumulating total_loss;

– DataLoader worker bloat: each worker ends up copying the large Python lists of the dataset, because of reference counting.

Consequence on a supercomputer: Slurm enforces a memory limit per job. When it is exceeded, the job is killed, often without a clear Python error message, but rather with a Slurm segfault that takes some deciphering.

Symptoms: process memory that grows over time up to the limit. You follow it during the job in CEEMS, and afterwards with Slurm's sacct (MaxRSS field).

Fixes: stream the data instead of loading everything, fix leaks with .item() or .detach(), store dataset metadata in NumPy arrays rather than Python lists, and adjust the number of workers.

Définition : c'est la mémoire du CPU, la RAM du nœud, qui est saturée.

Sur le schéma, on est cette fois côté CPU, dans sa DRAM. Trois causes classiques :

– charger ou mettre en cache tout le dataset en RAM ;

– une fuite mémoire dans la boucle, par exemple accumuler total_loss

– le gonflement des workers du DataLoader : chaque worker finit par recopier les grosses listes Python du dataset, à cause du comptage de références.

Conséquence sur un supercalculateur : Slurm impose une limite mémoire par job. Quand on la dépasse, le job est tué, souvent sans message d'erreur Python clair mais plutot un segfault slurm qu'il faut un peu déchiffrer par la suite

Symptômes : la mémoire du processus qui monte au fil du temps jusqu'à la limite. On la suit pendant le job dans CEEMS, et après coup avec sacct de slurm (champ MaxRSS).

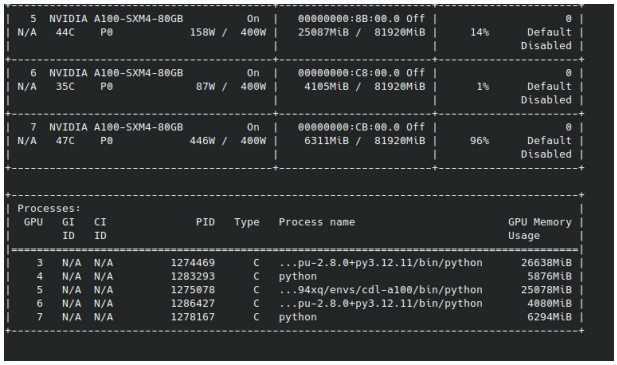
Corrections : lire les données en flux plutôt que tout charger, corriger les fuites avec .item() ou .detach(), stocker les métadonnées du dataset dans des tableaux NumPy plutôt que dans des listes Python, et ajuster le nombre de workers.

Tools

For each bottleneck, there is a better-suited tool. We present them from the simplest, with no instrumentation, to the finest, at kernel level. This is the order in which we use them in the lab.

Pour chaque goulot, il y a un outil plus adapté. On les présente du plus simple, sans instrumentation, au plus fin, au niveau du kernel. C'est l'ordre dans lequel on les utilise dans le TP.

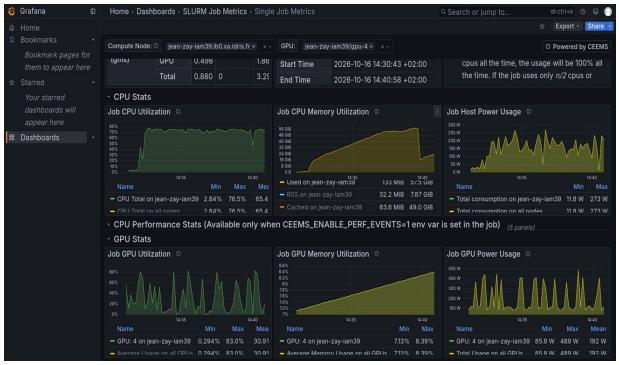
nvidia-smi & CEEMS



```
+-----------------------------------------+------------------------+----------------------+
|   5  NVIDIA A100-SXM4-80GB          On  |   00000000:8B:00.0 Off |                    0 |
| N/A   44C    P0             158W /  400W |   25087MiB /  81920MiB |     14%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+
|   6  NVIDIA A100-SXM4-80GB          On  |   00000000:C8:00.0 Off |                    0 |
| N/A   35C    P0              87W /  400W |    4105MiB /  81920MiB |      1%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+
|   7  NVIDIA A100-SXM4-80GB          On  |   00000000:CB:00.0 Off |                    0 |
| N/A   47C    P0             446W /  400W |    6311MiB /  81920MiB |     96%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+---------------------------------------------------------------------------------------+
| Processes:                                                                            |
|  GPU   GI   CI              PID   Type   Process name                  GPU Memory     |
|        ID   ID                                                         Usage          |
|=======================================================================================|
|    3   N/A  N/A         1274469      C   ...pu-2.8.0+py3.12.11/bin/python      26638MiB |
|    4   N/A  N/A         1283293      C   python                                5876MiB |
|    5   N/A  N/A         1275078      C   ...94xq/envs/cdl-a100/bin/python      25078MiB |
|    6   N/A  N/A         1286427      C   ...pu-2.8.0+py3.12.11/bin/python      4080MiB |
|    7   N/A  N/A         1278167      C   python                                6294MiB |
+---------------------------------------------------------------------------------------+
```

nvidia-smi: live view of the GPU, on the compute node

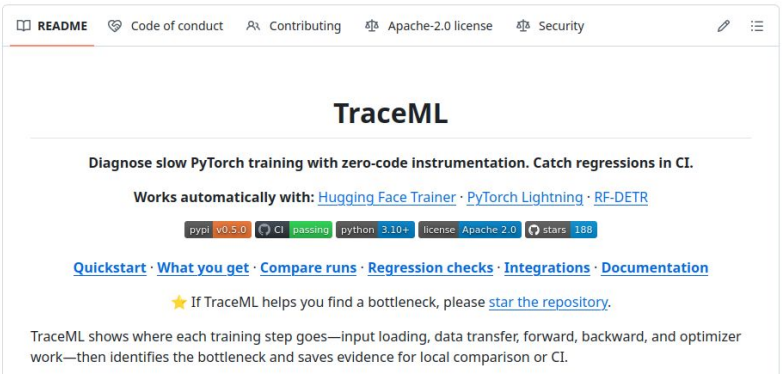


CEEMS: job history (CPU, GPU, memory, energy)

Best for

Is the GPU fed ? (4, 5) · 8 · VRAM · 9 · Host RAM

TraceML



Step-time breakdown and a verdict

Input wait, H2D, compute, memory per step

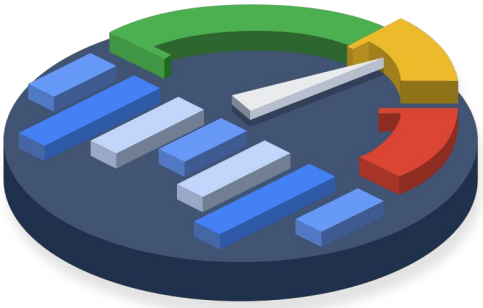
Best for

- **5 · Data loading**
- **10 · PCIe transfers**
- **8 · VRAM**
- **6 · Storage I/O (hint)**

https://traceopt.ai/

PyTorch Profiler





TensorBoard: operator and kernel views (legacy)

Perfetto: CPU and GPU timeline in the browser

Best for

1 · Compute | 4 · Launch | 5 · Data loading | 8 · VRAM

The PyTorch Profiler (torch.profiler) is the reference tool on the framework side. We enable it on a few iterations with a schedule: skip the first ones, do a warmup, then record. We tag our own phases with record_function.

It produces a single trace, readable by several viewers:

– Perfetto (ui.perfetto.dev): drag the .json file into the browser and you get the CPU and GPU timeline. This is where you see the gaps between kernels (launch), the waits at the start of a step (data), the Memcpy copies (PCIe).

– TensorBoard: its Operator and GPU Kernel views show, among other things, whether the Tensor Cores are used. Careful, the TensorBoard profiler plugin is deprecated: we keep it for these views, but Perfetto is the recommended way.

With profile_memory, you can also export a memory timeline of the VRAM as HTML. And the same trace can be analyzed by HTA for multi-GPU runs.

Important: NVTX tags are not shown here, they are for Nsight. For this profiler, use record_function.

Le PyTorch Profiler (torch.profiler) est l'outil de référence côté framework. On l'active sur quelques itérations avec un schedule : on ignore les premières, on fait un warmup, puis on enregistre. On balise nos propres phases avec record_function.

Il produit une seule trace, lisible par plusieurs visualiseurs :

– Perfetto (ui.perfetto.dev) : on glisse le fichier .json dans le navigateur et on obtient la timeline CPU et GPU. C'est là qu'on voit les trous entre les kernels (lancement), les attentes en début de pas (données), les copies Memcpy (PCIe).

– TensorBoard : ses vues Operator et GPU Kernel indiquent notamment si les Tensor Cores sont utilisés. Attention, le plugin TensorBoard du profiler est déprécié : on le garde pour ces vues, mais Perfetto est la voie recommandée.

Avec profile_memory, on peut aussi exporter une timeline mémoire en HTML pour la VRAM. Et la même trace peut être analysée par HTA en multi-GPU.

Point important : les balises NVTX ne s'affichent pas ici, elles servent à Nsight. Pour ce profiler, on utilise record_function.

NVIDIA Nsight Systems & Compute



Nsight Systems

Best for

4 · Launch | 7 · Comm



Nsight Compute

Best for

1 · Compute | 2 · Memory BW | 3 · Latency

The last level of the funnel: the NVIDIA tools. They are used from the command line, by wrapping the script launch (nsys profile python …, ncu …), then the report is opened in the graphical application on your workstation.

Nsight Systems (nsys) gives the timeline of the whole system: CUDA calls, kernels, memory copies, NCCL communications, and our NVTX tags. It is the tool for kernel-launch, inter-GPU communication and PCIe-transfer bottlenecks.

Nsight Compute (ncu) zooms in on one kernel at a time and replays it several times to read the hardware counters: occupancy, throughputs, Tensor Core usage, and the roofline. It is the only tool that decides between compute-bound, memory-bound and latency-bound. It is slow: use it only on a kernel already identified as expensive.

On a shared cluster, access to performance counters can be restricted for Nsight Compute. From Python, the Nsight Python library drives Nsight Compute (lab exercise).

Golden rule: go down the funnel from the simplest to the finest. Starting with Nsight Compute means looking at one tree while the forest is burning on the DataLoader side.

Le dernier niveau de l'entonnoir : les outils NVIDIA. Ils s'utilisent en ligne de commande, en enveloppant le lancement du script (nsys profile python …, ncu …), puis on ouvre le rapport dans l'application graphique sur son poste.

Nsight Systems (nsys) donne la timeline de tout le système : appels CUDA, kernels, copies mémoire, communications NCCL, et nos balises NVTX. C'est l'outil pour les goulots de lancement des kernels, de communication entre GPU et de transferts PCIe.

Nsight Compute (ncu) zoome sur un kernel à la fois et le rejoue plusieurs fois pour lire les compteurs matériels : occupancy, débits, utilisation des Tensor Cores, et le roofline. C'est le seul outil qui tranche entre compute-bound, memory-bound et latency-bound. Il est lent : on ne l'utilise que sur un kernel déjà identifié comme coûteux.

Sur un cluster partagé, l'accès aux compteurs de performance peut être restreint pour Nsight Compute. Pour un usage depuis Python, la bibliothèque Nsight Python pilote Nsight Compute (exercice du TP).

Règle d'or : on remonte l'entonnoir du plus simple au plus fin. Sortir Nsight Compute en premier, c'est regarder un arbre alors que la forêt brûle côté dataloader.

TP Time !

